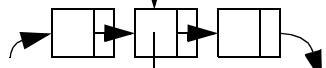
Education Data Structure

EDUCATION

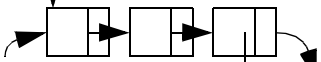
```
/* Usage */
EDUCATION *education_fetch(
        season_name,
        year,
        spreadsheet_filename,
        date_label );

/* Input */
SEMESTER *semester =
        semester_fetch(
            season_name,
            year,
            1 /* fetch_offering_list */,
            0 /* not fetch_registration_list */ );

PAYPAL *paypal =
        paypal_fetch(
            spreadsheet_filename,
            date_label );

LIST *education_program_list() =
        program_list(
            1 /* fetch_alias_list );

/* Process */
LIST *education_deposit_list(
        not_found_course_name_list,
        season_name,
        year,
        spreadsheet_filename,
        paypal->spreadsheet,
        paypal->paypal_dataset,
        semester_offering_list(),
        education_program_list(),
        education_product_list() );

PAYPAL_DATASET *education_paypal_dataset(
        input_string,
        paypal->
            spreadsheet->
            spreadsheet_column_list(),
        /* Returns paypal_dataset */
        paypal->paypal_dataset );

DEPOSIT *education_deposit(
        not_found_course_name_list /* out */,
        season_name,
        year,
        semester_offering_list(),
        education_program_list(),
        education_product_list(),
        paypal_dataset,
        row_number );

/* Output */
void education_deposit_insert(
        education_deposit_list() );

void education_payment_insert(
        education_deposit_list() );

void education_enrollment_insert(
        education_deposit_list() );
```

SEMESTER

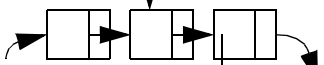
```
/* Usage */
SEMESTER *semester_new(
        char *season_name,
        int year );

/* Process */
LIST *semester_offering_list(
        season_name,
        year );

LIST *semester_event_list(
        season_name,
        year );
```



OFFERING

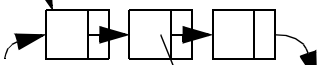


EVENT



DEPOSIT

PRODUCT



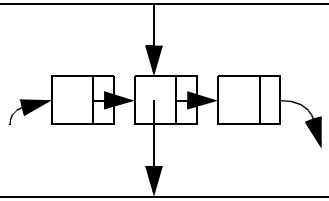
PROGRAM

PAYPAL

PAYPAL_DATASET

PAYPAL_DEPOSIT (1)

```
/* Usage */
PAYPAL_DEPOSIT *paypal_deposit_fetch(
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_tuition_payment_list,
        fetch_program_payment_list,
        fetch_tuition_refund_list,
        fetch_product_refund_list );

/* Input */
ENTITY *payor_entity =
        entity_new(
            payor_full_name,
            payor_street_address );

SEMESTER *semester =
        semester_new(
            season_name,
            year );

char *deposit_date_time;
double deposit_amount;
double transaction_fee;
double account_balance;
char *transaction_ID;
char *invoice_number;
char *from_email_address;

/* Process */
int row_number;

double deposit_net_revenue(
        deposit_amount,
        transaction_fee );

LIST *paypal_item_list =
paypal_item_list(
        paypal_item_entity_delimit(
            education_paypal_dataset()->
                item_title_P ),
        education_paypal_dataset()->
            transaction_type_E,
        education_paypal_allowed_list(
            offering_name_list()
            program_name_list(),
            program_alias_name_list(),
            product_name_list(),
            event_list() ) );

void deposit_set_paypal_item_expected_revenue(
        deposit_paypal_item_list(),
        semester_offering_list() );
```

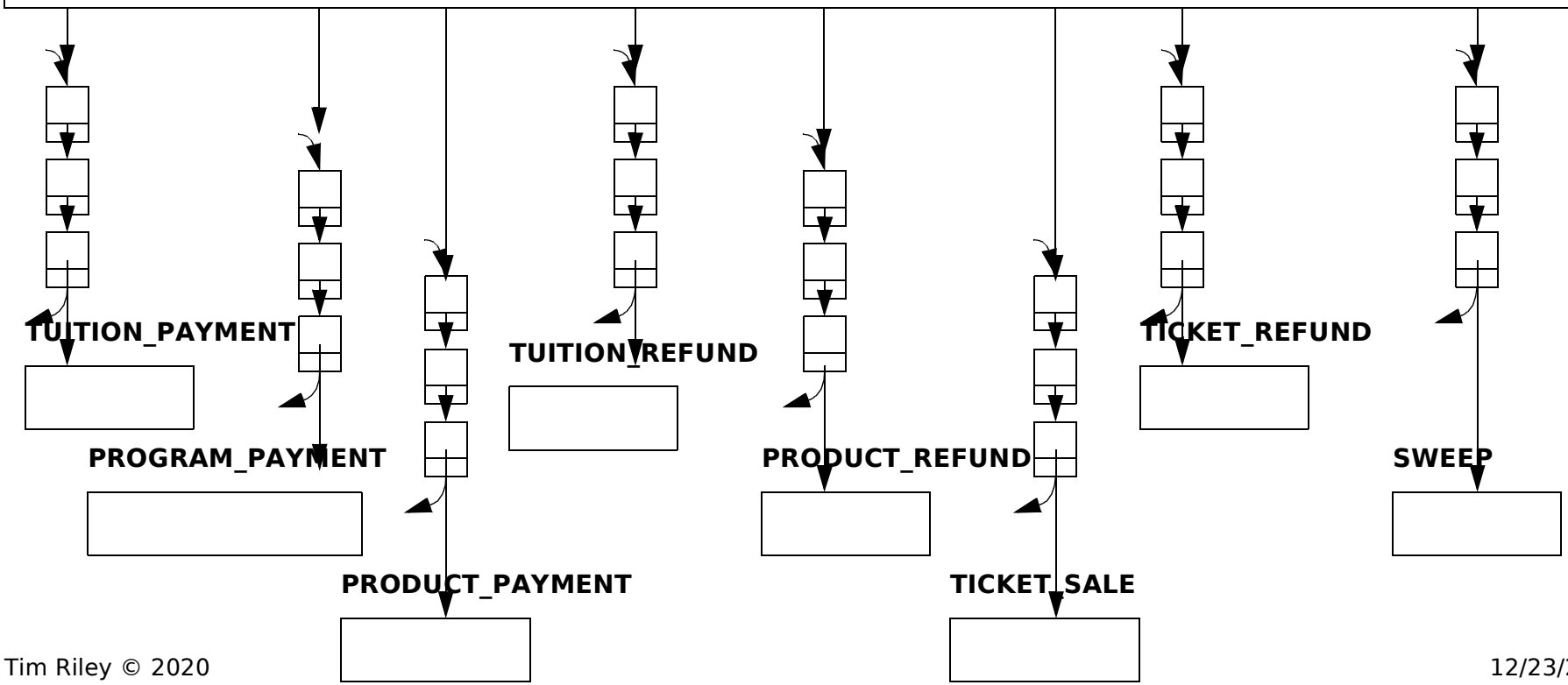


PAYPAL_ITEM

```
/* Usage */
LIST *paypal_item_list(
        not_exists_revenue_item_list,
        entity_delimited_item_title_P,
        transaction_type_E,
        allowed_item_list );

/* Input */
double expected_revenue;

/* Process */
char *item_data;
ENTITY *benefit_entity;

double paypal_item_value(
        deposit_amount,
        this->expected_revenue,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length()
);

double paypal_item_fee(
        deposit_amount,
        transaction_fee,
        paypal_item_value() );

double paypal_item_gain(
        deposit_amount,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );

LIST *paypal_item_steady_state_list(
        paypal_item_list,
        deposit_amount,
        transaction_fee,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );
```

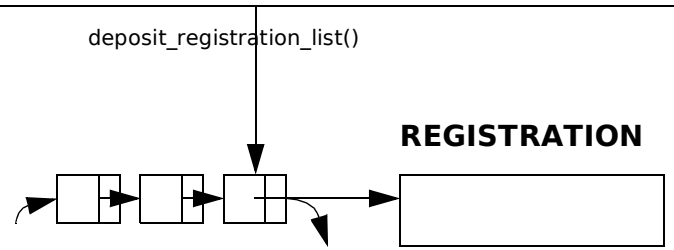
PAYPAL_DEPOSIT (2)

```
/* Process */

LIST *tuition_payment_list(
        season_name,
        year,
        paypal_item_steady_state_list(),
        semester_offering_list(),
        deposit );

LIST *program_payment_list(
        paypal_item_steady_state_list(),
        program_name_list(),
        program_alias_name_list(),
        deposit );

LIST *product_payment_list(
        paypal_item_steady_state_list(),
        product_name_list(),
        deposit );

LIST *tuition_refund_list(
        season_name,
        year,
        paypal_item_steady_state_list(),
        offering_name_list(),
        deposit );

LIST *product_refund_list(
        paypal_item_steady_state_list(),
        product_name_list(),
        deposit );

LIST *ticket_sale_list(
        paypal_item_steady_state_list(),
        event_list(),
        deposit );

LIST *ticket_refund_list(
        paypal_item_steady_state_list(),
        event_list(),
        deposit );

LIST *sweep_list(
        deposit );
```



PAYPAL_DEPOSIT (3)

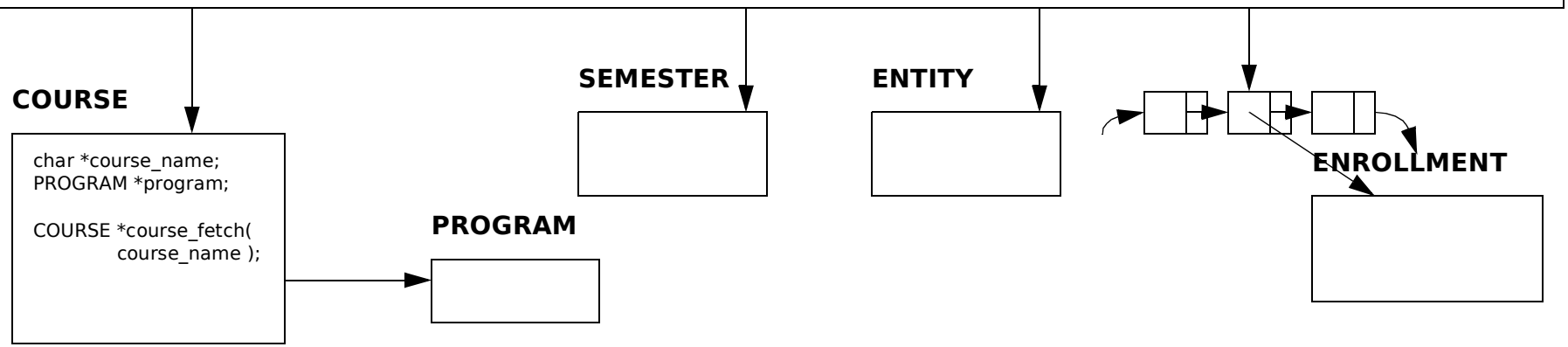
```
LIST *deposit_registration_list(
        deposit_tuition_payment_list(),
        deposit_tuition_refund_list() );

double tuition_payment_total(
        tuition_payment_list() );

double program_payment_total(
        program_payment_list() );

double product_payment_total(
        product_payment_list() );

double tuition_refund_total(
        tuition_refund_list() );

double product_refund_total(
        product_refund_list() );

double ticket_sale_total(
        ticket_sale_list() );

double ticket_refund_total(
        ticket_refund_list() );

double sweep_total(
        sweep_list() );

double deposit_registration_tuition(
        deposit_registration_list() )
{
        return
            sum(
            A->
            registration->
            registration_tuition() );
}

double deposit_gain_donation(
        deposit_amount,
        deposit_registration_tuition() );

double deposit_net_revenue(
        deposit_amount,
        transaction_fee )
{
        return A - B;
}

double deposit_overpayment_loss(
        deposit_amount,
        deposit_registration_tuition() );

DEPOSIT *deposit_steady_state(
        this,
        semester_offering_list );

void deposit_insert(
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        tuition_payment_total(),
        program_payment_total(),
        product_payment_total(),
        tuition_refund_total(),
        product_refund_total(),
        ticket_sale_total(),
        ticket_refund_total(),
        sweep_total(),
        transaction_fee );
```



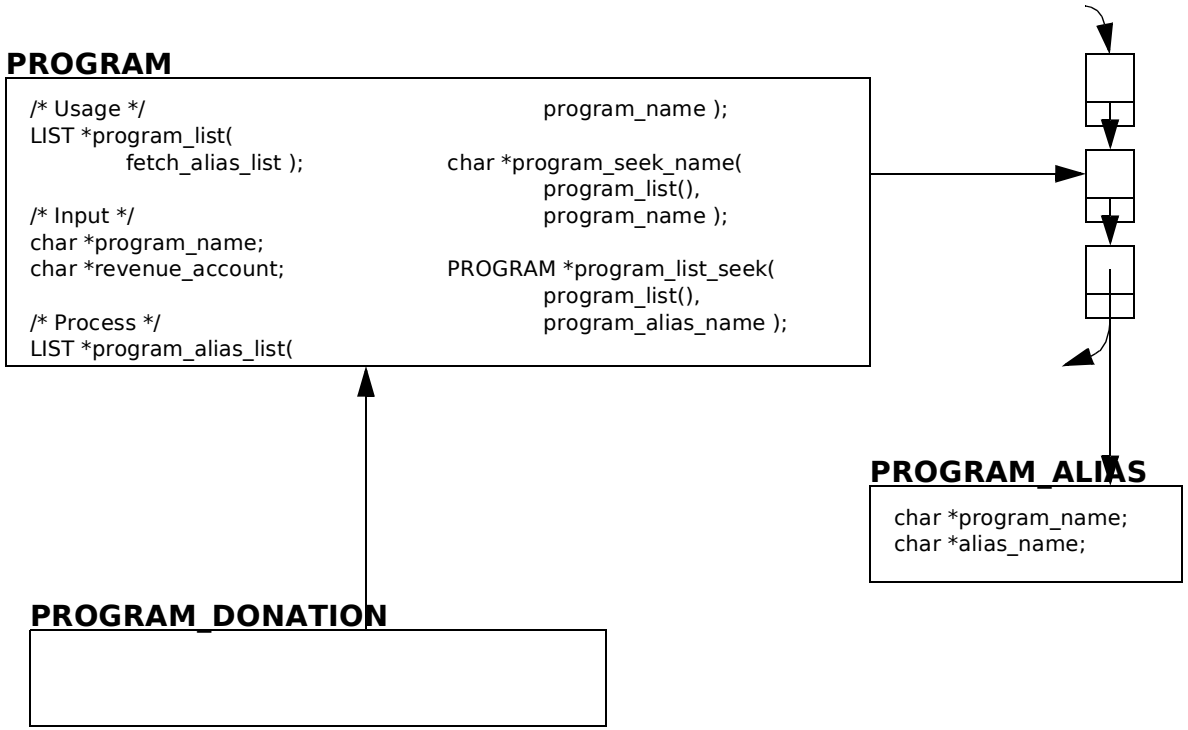
Education Data Structure

OFFERING

```
/* Usage */
OFFERING *offering_fetch(
        course_name,
        season_name,
        year,
        fetch_course,
        fetch_enrollment_list );

/* Input */
COURSE *course =
        course_fetch(
            course_name );

SEMESTER *semester =
        semester_new(
            season_name,
            year );

double course_price;
int class_capacity;
char *revenue_account;

ENTITY *instructor_entity =
        entity_new(
            instructor_full_name;
            instructor_street_address );

LIST *offering_enrollment_list(
            course_name,
            season_name,
            year );

/* Process */
OFFERING *offering_parse(
        input,
        fetch_course,
        fetch_enrollment_list )

int offering_enrollment_count(
        offering_enrollment_list() )
{
        return length( A );
}

int offering_capacity_available(
        class_capacity,
        offering_enrollment_count() )
{
        return A - B;
}

OFFERING *offering_steady_state(
        this,
        offering_enrollment_list() );

/* Output */
void offering_update(
        offering_enrollment_count(),
        offering_capacity_available(),
        course_name,
        season_name,
        year );
```



Education Data Structure



PROGRAM

PROGRAM_DONATION

```
/* Usage */
PROGRAM_DONATION *program_donation_fetch(
        program_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        payment_date_time,
        fetch_program,
        fetch_paypal );

PROGRAM_PAYMENT *program_payment_steady_state(
        this,
        deposit_amount,
        transaction_fee );

/* Input */
PROGRAM *program;
double donation_amount;
double fees_expense;

/* Process */
double program_donation_net_payment_amount(
        donation_amount,
        fees_expense )
{
        return A - B;
}

PROGRAM_PAYMENT *program_payment_parse(
        input,
        fetch_program,
        fetch_paypal );

TRANSACTION *program_donation_transaction(
        payor_full_name,
        payor_street_address,
        payment_date_time,
        program_name,
        payment_amount,
        fees_expense,
        program_donation_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        program->revenue_account );

/* Output */
void program_payment_update(
        program_donation_net_payment_amount(),
        program_donation_transaction()->
            transaction_date_time,
        program_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        payment_date_time );
```

TRANSACTION

PRODUCT

PRODUCT_PAYMENT

```
/* Usage */
PRODUCT_PAYMENT *product_payment_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_product,
        fetch_deposit );

PRODUCT_PAYMENT *product_payment(
        item_title_P,
        product_number,
        education_product_list,
        deposit );

/* Input */
PRODUCT *product;
DEPOSIT *deposit;

/* Process */
char *item_title_name =
        item_title_product_name(
            item_title_P,
            program_number
                /* always one */,
            education_product_list() );

double product_payment_amount(
        deposit->deposit_amount )
{
        return A;
}

double product_payment_fees_expense(
        deposit->transaction_fee )
{
        return A;
}

double product_payment_net_payment_amount(
        deposit->deposit_amount,
        deposit->transaction_fee )
{
        return A - B;
}

PRODUCT_PAYMENT *product_payment_parse(
        input,
        fetch_product,
        fetch_deposit );

PRODUCT_PAYMENT *
product_payment_steady_state(
        this,
        deposit->deposit_amount,
        deposit->transaction_fee,
        deposit->net_payment_amount );

TRANSACTION *product_payment_transaction(
        payor_full_name,
        payor_street_address,
        deposit_date_time,
        product_name,
        program_name,
        product_payment_amount(),
        product_fees_expense(),
        product_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account;

/* Output */
void product_payment_update(
        product_payment_amount(),
        product_payment_fees_expense(),
        product_payment_net_payment_amount(),
        product_payment_transaction()->
            transaction_date_time,
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time );
```

DEPOSIT

```
double deposit_amount;
double fees_expense;
double net_payment_amount;
LIST *product_payment_list();
```

TRANSACTION

PRODUCT

PRODUCT_REFUND

```
/* Usage */
PRODUCT_REFUND *product_refund_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_product,
        fetch_deposit );

PRODUCT_REFUND *product_refund(
        item_title_P,
        product_number,
        education_product_list,
        deposit );

/* Input */
PRODUCT *product;
DEPOSIT *deposit;

/* Process */
char *item_title_name =
        item_title_product_name(
            item_title_P,
            program_number
                /* always one */,
            education_product_list() );

double product_refund_amount(
        deposit->deposit_amount )
{
        return abs( A );
}

double product_refund_fees_expense(
        deposit->transaction_fee )
{
        return A;
}

double product_refund_net_payment_amount(
        deposit->deposit_amount,
        deposit->transaction_fee )
{
        return A - B;
}

PRODUCT_REFUND *product_refund_parse(
        input,
        fetch_product,
        fetch_deposit );

PRODUCT_REFUND *
product_refund_steady_state(
        this,
        deposit->deposit_amount,
        deposit->transaction_fee,
        deposit->net_payment_amount );

TRANSACTION *product_refund_transaction(
        payor_full_name,
        payor_street_address,
        deposit_date_time,
        product_name,
        program_name,
        product_refund_amount(),
        product_refund_fees_expense(),
        product_refund_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account;

/* Output */
void product_refund_update(
        product_refund_amount(),
        product_refund_fees_expense(),
        product_refund_net_payment_amount(),
        product_refund_transaction()->
            transaction_date_time,
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time );
```

DEPOSIT

```
double deposit_amount;
double fees_expense;
double net_payment_amount;
LIST *product_refund_list();
```

TRANSACTION

EVENT

TICKET_SALE

```
/* Usage */
PRODUCT_PAYMENT *product_payment_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_product,
        fetch_deposit );

PRODUCT_PAYMENT *product_payment(
        item_title_P,
        product_number,
        education_product_list,
        deposit );

/* Input */
PRODUCT *product;
DEPOSIT *deposit;

/* Process */
char *item_title_name =
        item_title_product_name(
            item_title_P,
            program_number
                /* always one */,
            education_product_list() );

double product_payment_amount(
        deposit->deposit_amount )
{
        return A;
}

double product_payment_fees_expense(
        deposit->transaction_fee )
{
        return A;
}

double product_payment_net_payment_amount(
        deposit->deposit_amount,
        deposit->transaction_fee )
{
        return A - B;
}

PRODUCT_PAYMENT *product_payment_parse(
        input,
        fetch_product,
        fetch_deposit );

PRODUCT_PAYMENT *
product_payment_steady_state(
        this,
        deposit->deposit_amount,
        deposit->transaction_fee,
        deposit->net_payment_amount );

TRANSACTION *product_payment_transaction(
        payor_full_name,
        payor_street_address,
        deposit_date_time,
        product_name,
        program_name,
        product_payment_amount(),
        product_fees_expense(),
        product_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account;

/* Output */
void product_payment_update(
        product_payment_amount(),
        product_payment_fees_expense(),
        product_payment_net_payment_amount(),
        product_payment_transaction()->
            transaction_date_time,
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time );
```

DEPOSIT

```
double deposit_amount;
double fees_expense;
double net_payment_amount;
LIST *product_payment_list();
```

TRANSACTION

EVENT

TICKET_REFUND

```
/* Usage */
PRODUCT_REFUND *product_refund_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time,
        fetch_product,
        fetch_deposit );

PRODUCT_REFUND *product_refund(
        item_title_P,
        product_number,
        education_product_list,
        deposit );

/* Input */
PRODUCT *product;
DEPOSIT *deposit;

/* Process */
char *item_title_name =
        item_title_product_name(
            item_title_P,
            program_number
                /* always one */,
            education_product_list() );

double product_refund_amount(
        deposit->deposit_amount )
{
        return abs( A );
}

double product_refund_fees_expense(
        deposit->transaction_fee )
{
        return A;
}

double product_refund_net_payment_amount(
        deposit->deposit_amount,
        deposit->transaction_fee )
{
        return A - B;
}

PRODUCT_REFUND *product_refund_parse(
        input,
        fetch_product,
        fetch_deposit );

PRODUCT_REFUND *
product_refund_steady_state(
        this,
        deposit->deposit_amount,
        deposit->transaction_fee,
        deposit->net_payment_amount );

TRANSACTION *product_refund_transaction(
        payor_full_name,
        payor_street_address,
        deposit_date_time,
        product_name,
        program_name,
        product_refund_amount(),
        product_refund_fees_expense(),
        product_refund_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account;

/* Output */
void product_refund_update(
        product_refund_amount(),
        product_refund_fees_expense(),
        product_refund_net_payment_amount(),
        product_refund_transaction()->
            transaction_date_time,
        product_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        deposit_date_time );
```

DEPOSIT

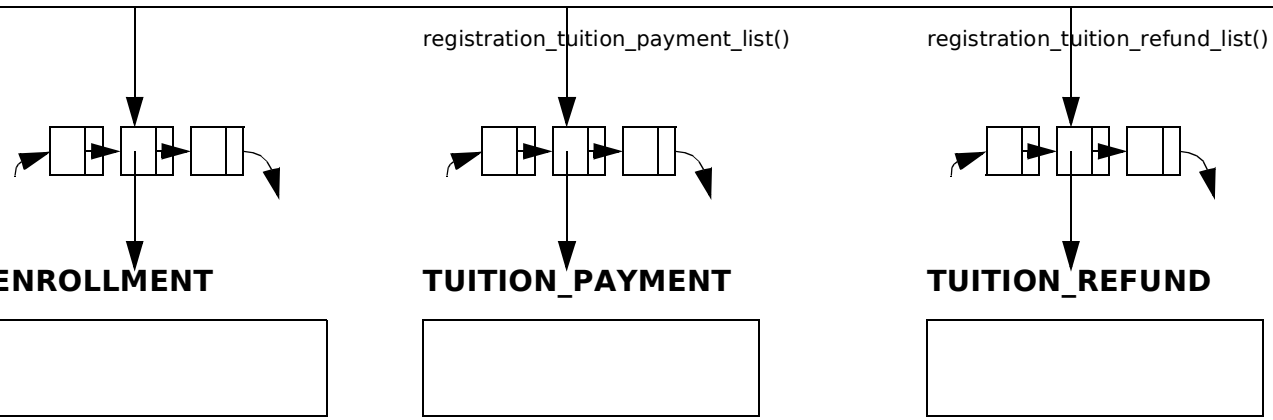
```
double deposit_amount;
double fees_expense;
double net_payment_amount;
LIST *product_refund_list();
```

TRANSACTION

Education Data Structure

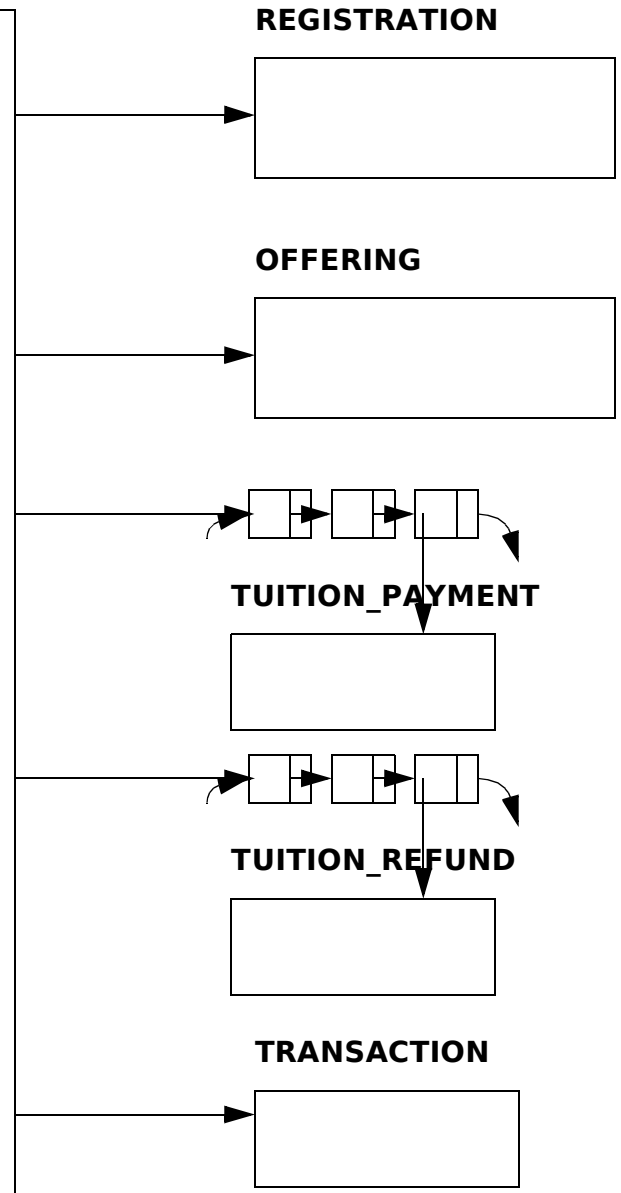
REGISTRATION

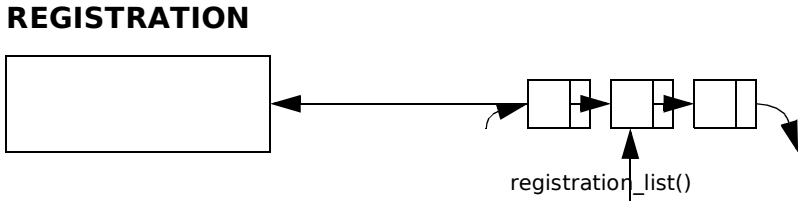
```
/* Usage */
REGISTRATION *registration_fetch(
                student_full_name,
                student_street_address,
                season_name,
                year,
                fetch_enrollment_list );

/* Input */
char *student_full_name;
char *street_address;
char *season_name;
int year;

/* Process */
LIST *registration_enrollment_list(
          student_full_name,
          street_address,
          season_name,
          year );

double registration_tuition(
          registration_enrollment_list(),
          semester_offering_list() );

LIST *registration_tuition_payment_list(
          registration_enrollment_list() );

double registration_tuition_payment_total(
          registration_tuition_payment_list )
{
          return sum(A->
                    tuition_payment->
                    payment_amount );
}

double registration_invoice_amount_due(
          registration_tuition(),
          registration_tuition_payment_total(),
          registration_tuition_refund_total() );

LIST *registration_tuition_refund_list(
          registration_enrollment_list() );

double registration_tuition_refund_total(
          registration_tuition_refund_list() );

void registration_update(
          registration_tuition(),
          registration_tuition_payment_total(),
          registration_invoice_amount_due(),
          student_full_name,
          student_street_address,
          season_name,
          year );
```



ENROLLMENT

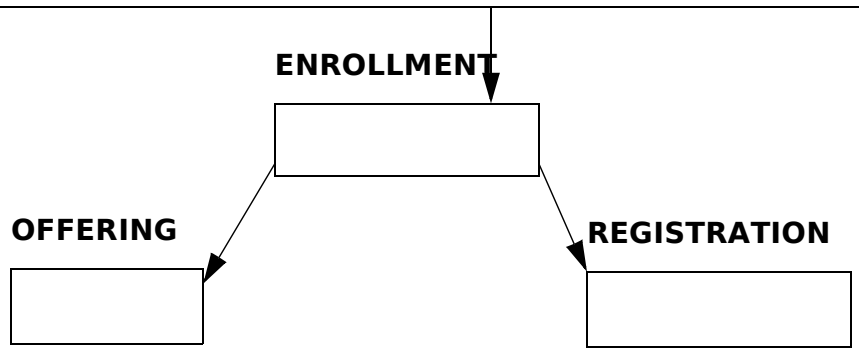
```
/* Usage */
ENROLLMENT *enrollment_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        fetch_tuition_payment_list,
        fetch_offering,
        fetch_registration,
        fetch tuition_refund_list );

/* Input */
LIST *enrollment_tuition_payment_list(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

LIST *enrollment_tuition_refund_list(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

OFFERING *offering =
        offering_new(
            course_name,
            season_name,
            year );

REGISTRATION *registration =
        registration_fetch(
            student_full_name,
            street_address,
            season_name,
            year,
            0 /* not fetch_enrollment_list */ );

/* Process */
double enrollment_tuition_payment_total(
        enrollment_tuition_payment_list() );

double enrollment_tuition_refund_total(
        enrollment_tuition_refund_list() );

char *enrollment_primary_where(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

TRANSACTION *enrollment_transaction(
        student_full_name,
        street_address,
        this->registration->registration_date_time,
        this->offering->course->program_name,
        this->offering->course->course_price,
        account_receivable(),
        offering_revenue_account() );

void enrollment_cancelled_date_update(
        update_pipe,
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        cancelled_date );
```





TUITION_PAYMENT (1)

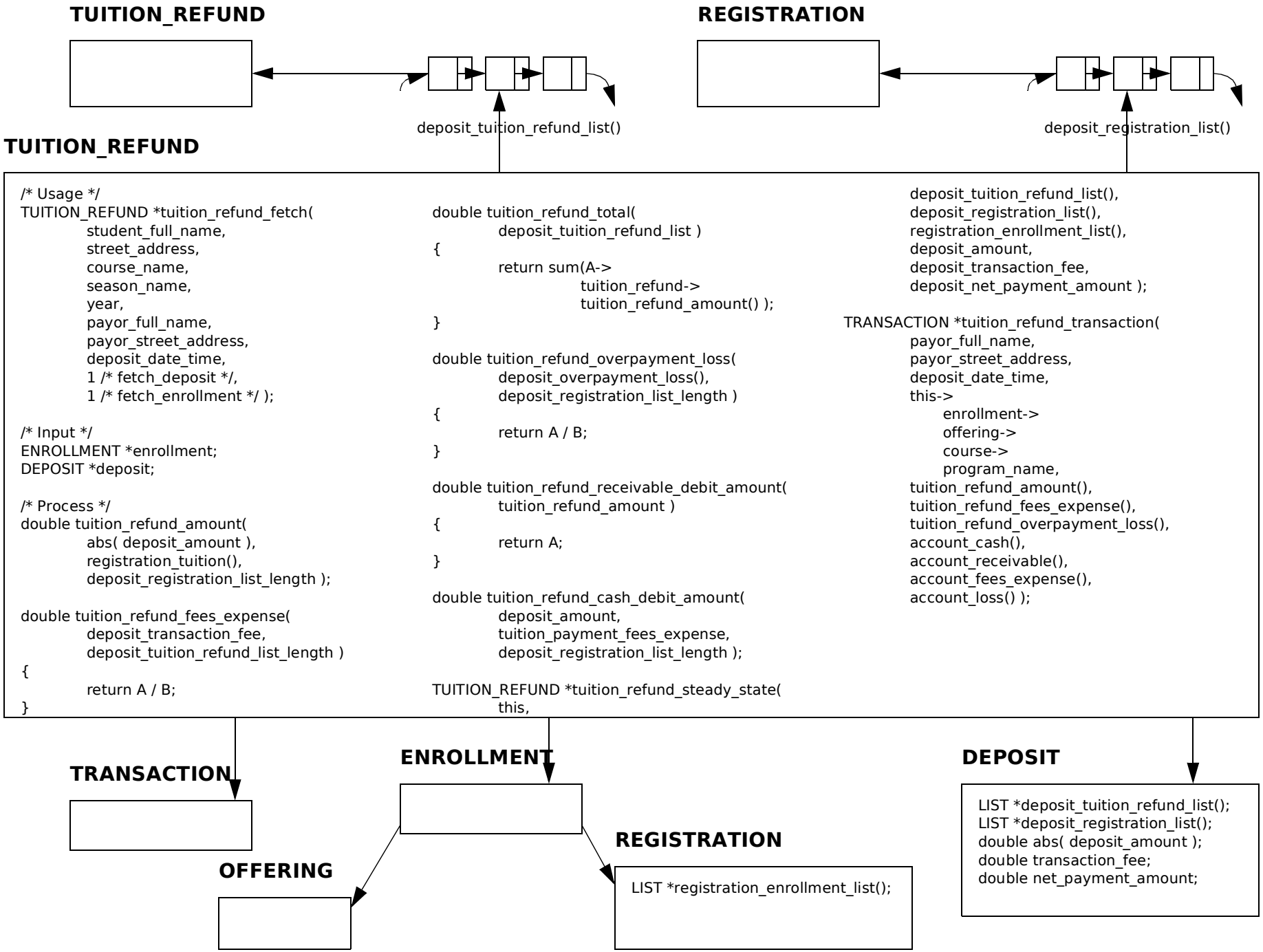
```
/* Usage */
TUITION_PAYMENT *tuition_payment_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time,
        0 /* not fetch_paypal */,
        1 /* fetch_enrollment */ );

TUITION_PAYMENT *tuition_payment_steady_state(
        this,
        tuition_payment_amount,
        fees_expense,
        registration_list() );

/* Input */
ENROLLMENT *enrollment;
double tuition_payment_amount;
double fees_expense;

/* Process */
double tuition_payment_net_payment_amount(
        tuition_payment_amount,
        fees_expense )
{
        return A - B;
}

double tuition_payment_total(
        tuition_payment_list )
{
        return sum(A->
                tuition_payment->
                tuition_payment_amount );
}

double tuition_payment_overpayment_donation(
        tuition_payment_amount,
        registration_list() );

double tuition_payment_receivable_credit_amount(
        tuition_payment_amount )
{
        return A;
}

double tuition_payment_cash_debit_amount();
```



TUITION_PAYMENT (2)

```
TRANSACTION *tuition_payment_transaction(
        payor_full_name,
        payor_street_address,
        payment_date_time,
        this->
            enrollment->
            offering->
            course->
            program_name,
        tuition_payment_amount,
        fees_expense,
        tuition_payment_net_payment_amount(),
        tuition_payment_overpayment_donation(),
        account_cash(),
        account_receivable(),
        account_fees_expense(),
        account_gain() );
```

```
void tuition_payment_update(
        tuition_payment_net_payment_amount(),
        tuition_payment_overpayment_donation(),
        tuition_payment_transaction()->
            transaction_date_time,
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time );
```

TRANSACTION

Education Data Structure